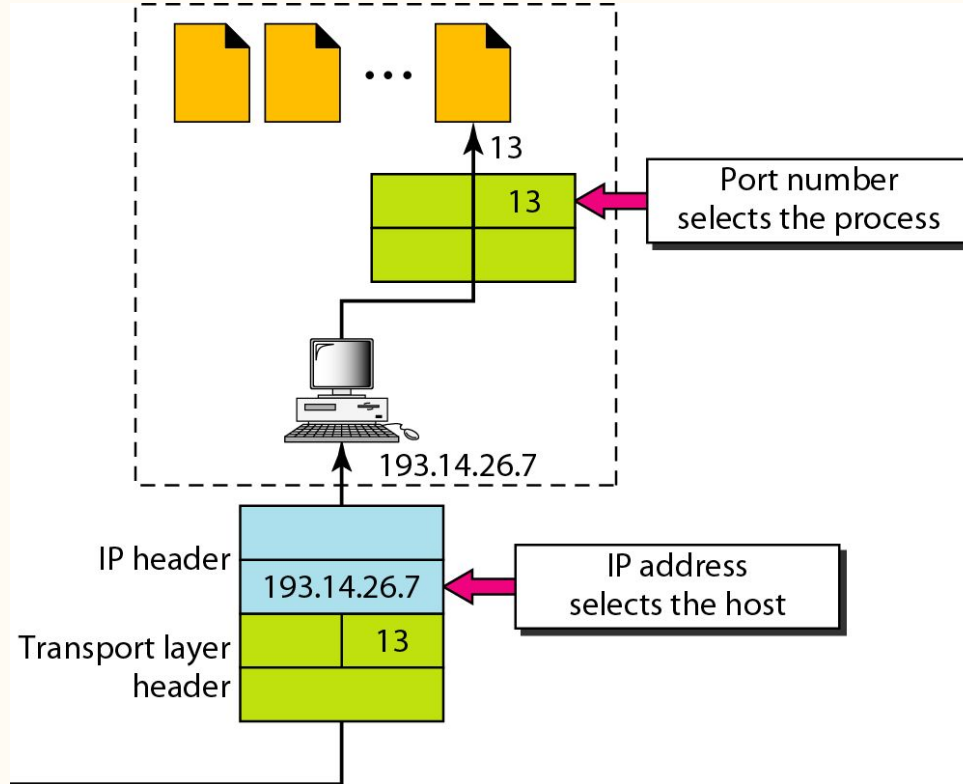


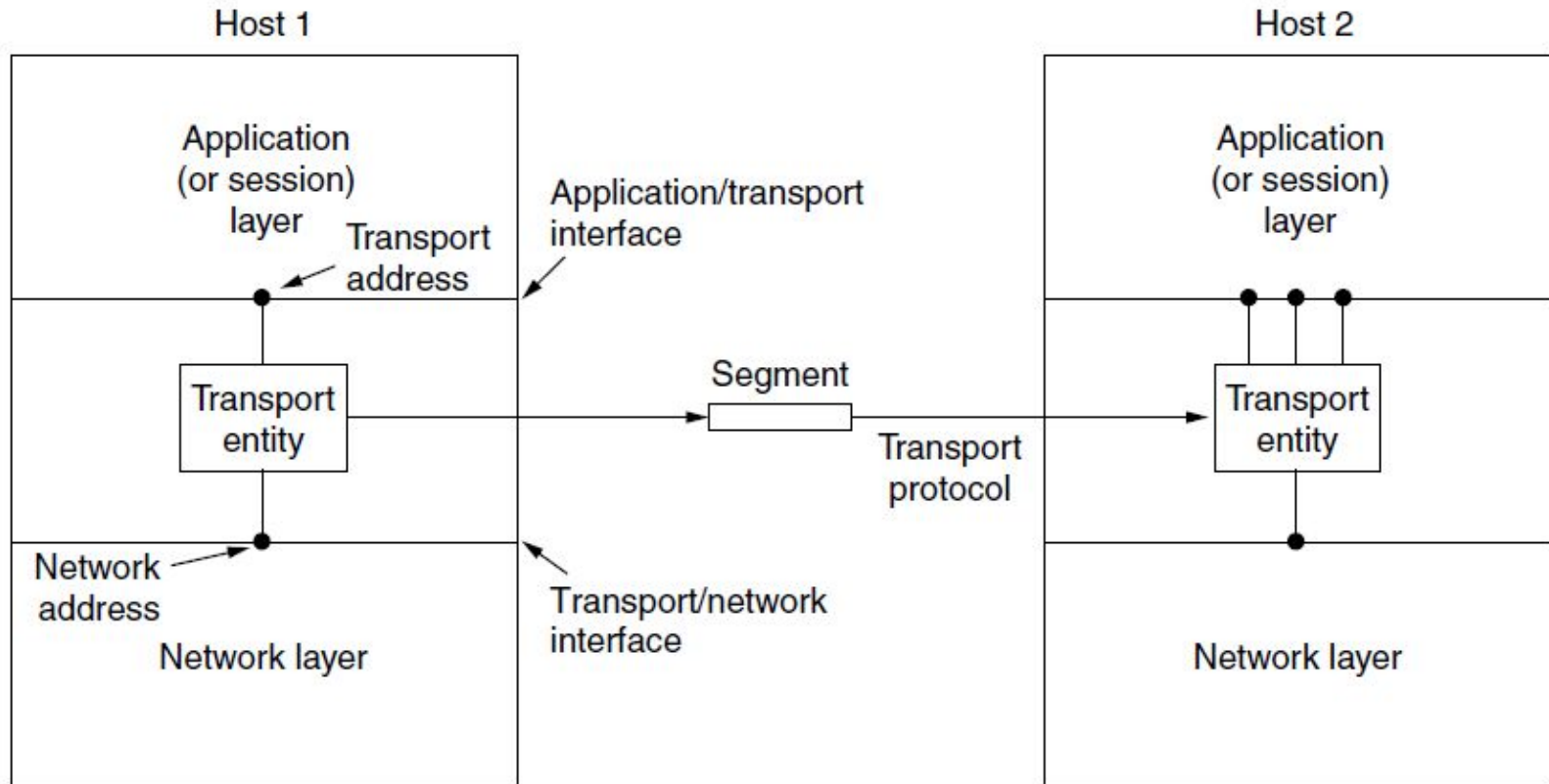
Transport Layer

Er. Sudarshan Devkota

Transport Layer



Transport Layer



Functions of Transport Layer

1. Segmentation and Reassembly

Application data is often too large to be sent as a single unit across a network. The Transport Layer breaks these large data streams into manageable chunks called Segments (TCP) or Datagrams (UDP).

Each segment is assigned a sequence number, allowing the receiving Transport Layer to reorder packets if they arrive out of sequence and reconstruct the original data stream perfectly.

2. Service Point Addressing (Port Addressing)

While the Network Layer uses IP addresses to reach a specific host, the Transport Layer uses Port Numbers to deliver data to the correct process or application running on that host.

A single computer might have a web server, an email client, and a gaming application open simultaneously. Port numbers (e.g., 80 for HTTP, 25 for SMTP) act as unique identifiers that ensure incoming data reaches the specific program that requested it.

Functions of Transport Layer

3. Connection Control

The Transport Layer manages the connection state between two endpoints.

- *Connection-Oriented (TCP)*: Requires a formal "three-way handshake" (SYN, SYN-ACK, ACK) before data transfer begins. This ensures both sender and receiver are ready and synchronized.
- *Connectionless (UDP)*: Sends data without pre-establishing a connection. It is faster but less reliable, as it does not guarantee delivery.

4. Flow Control

This mechanism prevents a fast sender from overwhelming a slower receiver. The receiver continuously reports its "receive window" size (the amount of buffer space it has left) to the sender. The sender adjusts its transmission rate accordingly, ensuring the receiver is never forced to drop packets due to buffer overflow.

Functions of Transport Layer

5. Error Control and Reliability

This ensures that data is delivered accurately and without corruption.

Checksums: Each segment contains a checksum to detect bit-level corruption.

Acknowledgements (ACKs): The receiver sends an ACK to confirm that a segment was received successfully.

Retransmission: If the sender does not receive an ACK within a set timeout, it assumes the packet was lost or corrupted and retransmits it.

Ports

A port is a 16-bit logical address used by the Transport Layer (TCP/UDP) to identify a specific process or service on a host.

Without ports, a computer would not be able to distinguish between an incoming email, a web page request, and a file transfer occurring at the same time.

Port numbers range from 0 to 65,355.

1. **Well-Known Ports (0 - 1023):** These are reserved for standard, universal services and are assigned by IANA (Internet Assigned Numbers Authority). *Eg. HTTP (80), HTTPS (443), FTP (21), and DNS (53).*
2. **Registered Ports (1024 - 49151):** These are used for specific applications and software companies that register them with IANA. *Eg. MySQL Database (3306), Microsoft SQL Server (1433).*
3. **Dynamic or Private Ports (49152 - 65535):** Also known as "Ephemeral Ports," these are assigned temporarily by the operating system when a client initiates a connection to a server.

Elements of Transport Protocols

1. Addressing (TSAPs, NSAPs, and Port Binding):

Transport protocols must distinguish between multiple processes running on the same host. This is achieved through a layered addressing hierarchy.

- NSAP (Network Service Access Point): This is the IP Address. It identifies the physical host on the network.
- TSAP (Transport Service Access Point): This is the Port Number. It identifies a specific application (like a web server) inside that host.
- When a process wants to communicate, it "listens" on a specific TSAP. For a connection to occur, a Transport Entity must map the TSAP to an NSAP to find the route.
- The combination of an IP address and a Port number is called a Socket. A unique connection is defined by two sockets (Source and Destination).

Elements of Transport Protocols

2. Connection Establishment (Solving the "Delayed Duplicate" Problem)

Establishing a connection is surprisingly difficult because the network can "hold" packets and deliver them much later, potentially confusing the receiver into opening a fake or "zombie" connection.

To prevent delayed duplicate packets from causing chaos, we use Sequence Numbers that are tied to a clock or a pseudo-random generator.

1. Host 1 sends a request with sequence x.
2. Host 2 responds, acknowledging x and proposing its own sequence y.
3. Host 1 acknowledges y and sends the first byte of data.

Even if an old, delayed "Connection Request" arrives at Host 2 years later, the handshake will fail at the third step because Host 1 will realize the sequence numbers don't match its current state.

Elements of Transport Protocols

3. Connection Release (The Two-Army Problem)

Releasing a connection is a "consensus" problem. If one side hangs up abruptly (Asymmetric Release), data currently in transit will be lost.

The connection is treated as two independent one-way links.

1. Host A sends a FIN (Finished) packet.
2. Host B acknowledges. Host A stops sending but can still receive.
3. Once Host B is finished, it sends its own FIN.
4. Host A acknowledges. The connection is officially dead.

Technically, it is impossible to be 100% certain both sides know the connection is closed if the final acknowledgment is lost. Protocols use Timers to force a shutdown if a response isn't received within a reasonable window.

Elements of Transport Protocols

4. Flow Control and Buffering

Flow control ensures the sender doesn't drown the receiver in data. This is handled via Windowing.

Sender Side: Must buffer all transmitted data until an ACK is received, in case retransmission is needed.

Receiver Side: Buffers out-of-order packets to reassemble them into the correct sequence before handing them to the application.

The receiver doesn't just say "I got it." It says "I got it, and I have room for 4000 more bytes." This is the Sliding Window. If the receiver is busy, it shrinks the window to zero, effectively hitting a "pause" button on the sender.

Elements of Transport Protocols

5. Multiplexing and De-multiplexing

This is the ability to run multiple network applications over a single physical wire.

Upward Multiplexing: Many transport connections share one network-layer connection (IP). This is standard for home users where one IP handles Netflix, Zoom, and Gaming simultaneously.

Downward Multiplexing: One transport connection is split across multiple network paths. This is used in high-end servers to increase throughput or provide redundancy (if one path fails, the data keeps moving).

Elements of Transport Protocols

6. Error Control (Checksums and Timers)

The Transport Layer must ensure the data is bit-perfect.

Checksums: A mathematical value is calculated based on the packet data. If the receiver calculates a different value, the packet is discarded as corrupted.

Retransmission Timers: Every time a packet is sent, a timer starts. If an ACK doesn't return before the timer hits zero, the packet is presumed "lost at sea" and is resent.

Elements of Transport Protocols

7. Crash Recovery (The "Resurrection" Problem)

What happens if a server crashes and reboots in the middle of a transfer?

The server has lost its memory of what it already acknowledged.

Strategies:

First-ACK: The server sends an ACK before writing data to the application.
(Risky: data could be lost in the crash).

Last-ACK: The server sends an ACK after successful processing. (Safer, but leads to more retransmissions).

Recovery is often handled by the Application Layer (e.g., your browser realizes the connection died and asks the server "Where did we leave off?").

User Datagram Protocol (UDP)

UDP is the simplest transport layer protocol in the Internet Protocol suite.

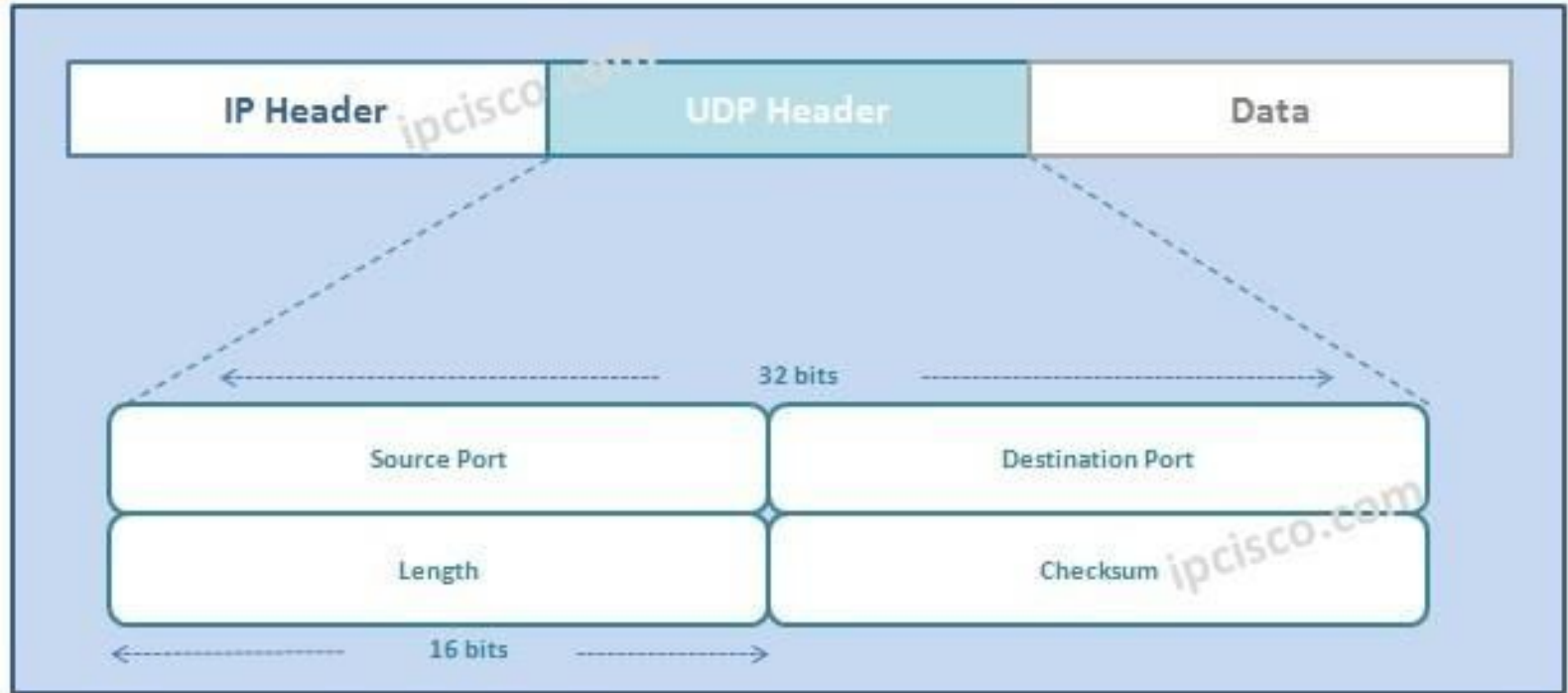
It provides a "best-effort" delivery service, meaning it sends data without verifying if the receiver is ready or if the data actually arrived.

UDP is a connectionless, unreliable transport protocol defined in RFC 768.

It adds the absolute minimum overhead to IP. Essentially, it just provides Port Numbers so that data reaches the right application and an optional Checksum to ensure data integrity.

UDP does not use handshakes, acknowledgments, or retransmissions. If a packet is lost in the network, UDP does not care; the application must handle the loss.

User Datagram Protocol (UDP)



User Datagram Protocol (UDP)

The UDP unit of data is called a User Datagram. It has a very small, fixed-size header of only 8 bytes, compared to the 20-byte minimum of TCP.

Source Port (16 bits): Identifies the sending process.

Destination Port (16 bits): Identifies the receiving process.

Length (16 bits): Indicates the total length of the user datagram (header + data).

Checksum (16 bits): A mathematical value used to detect bit-errors in the header and data. If the checksum is invalid, the receiver discards the packet.

UDP Operations

UDP operation is straightforward and characterized by a lack of "state."

Connectionless Communication: No "Three-Way Handshake" is required. A sender can start blasting data immediately to a destination IP and Port.

No Flow Control: UDP does not check if the receiver's buffer is full. It sends data at the rate the application provides.

No Error Recovery: There are no sequence numbers. If packets arrive out of order, UDP delivers them to the application out of order. If a packet is missing, it stays missing.

Full Independence: Every UDP datagram is an independent entity. There is no relationship between Datagram A and Datagram B, even if they are part of the same stream.

Uses of UDP

Real-Time Streaming & VoIP: In a Discord call or a YouTube Live stream, a dropped frame is better than a 2-second delay caused by TCP retransmitting a lost packet.

Query/Response Services (DNS): When you type a URL, your computer sends a small DNS query. If it doesn't get an answer, it just asks again. A handshake would double the time it takes to load a page.

Routing Protocols (RIP/SNMP): Network management tools use UDP because they need to be lightweight and functional even when the network is congested.

Multicasting & Broadcasting: UDP supports sending one packet to many people simultaneously, which is mathematically difficult for a connection-oriented protocol like TCP.

DHCP: Assigning IP addresses to new devices requires broadcasting, which UDP handles naturally.

RPC

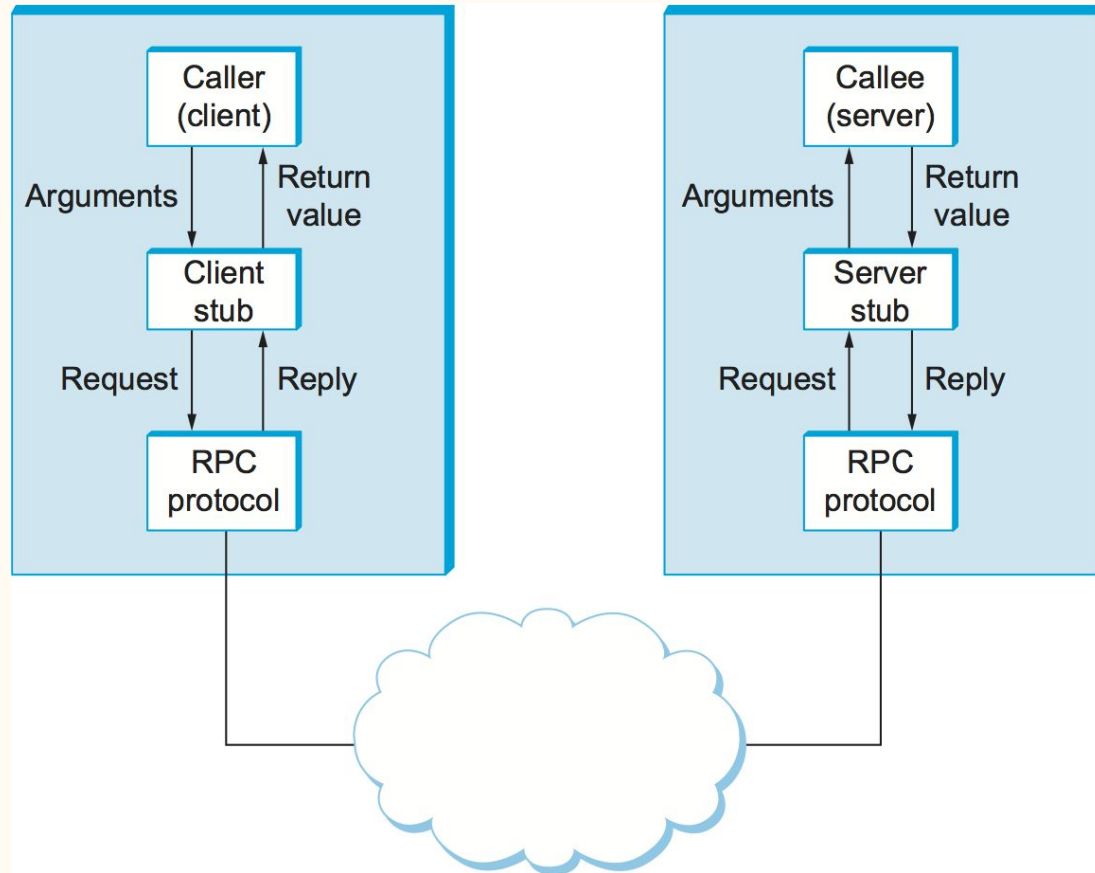
The Traditional View: Networking is often viewed as "Message Passing"—moving a set of bytes from Point A to Point B.

The RPC View: Networking should look like a Local Procedure Call (LPC).

RPC is a mechanism that allows a program to execute a function on a remote machine as if it were a local function, hiding all the complexities of the underlying network.

RPC creates a high-level communication model that sits on top of the transport layer. Instead of managing sockets and byte streams, developers call functions.

RPC



RPC

The Workflow:

Client Stub: Acts as a proxy on the client machine. It packs parameters into a message (Marshaling).

Transport: The message is sent across the network.

Server Stub: Receives the packet on the server, unpacks the data (Unmarshaling), and executes the actual code.

Return: The result travels back through the same path.

SunRPC

SunRPC (Open Network Computing RPC) is a communication framework developed by Sun Microsystems that allows a program to call functions on a remote server as if they were local. It uses XDR (External Data Representation) to ensure data is understood by different hardware and a Portmapper (running on UDP Port 111) to help clients find the correct service ports.

NFS (Network File System) is the primary application that utilizes SunRPC over UDP because it prioritizes speed and resilience for file access

Because UDP is connectionless, NFS doesn't track "active sessions." If an NFS server crashes and reboots, it immediately starts processing new UDP packets again. The client experiences no "connection error" it just waits for the server to reply.

File operations (Read/Write) are sent as independent UDP datagrams. By skipping connection setup, NFS performs remote file operations with minimal delay.

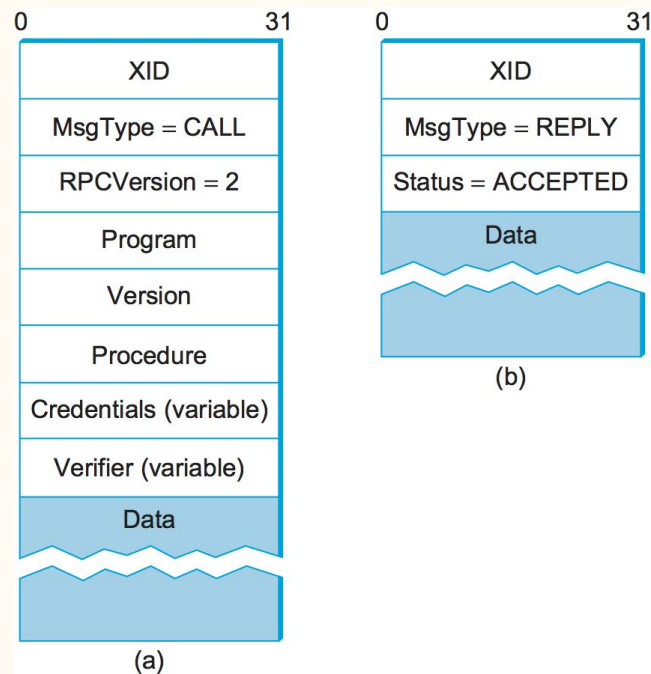
SunRPC

Since UDP can drop packets, SunRPC handles reliability itself. If a response isn't received, the RPC client uses a Timer to re-send the UDP request. To prevent duplicate actions, every request has a unique Transaction ID (XID), which the server checks to ensure it only processes a request once.

Figure 142. SunRPC header formats:

(a) request;

(b) reply.



Building Reliable Data Transfer Protocol

Reliable Data Transfer (RDT) defines the mechanisms used to provide a reliable logical channel over an inherently unreliable network layer (IP).

The complexity of an RDT protocol is defined by the characteristics of the underlying channel. We build the protocol in stages to address specific network failures:

1. dt 1.0: Transfer over a Reliable Channel:

Assumes an underlying channel that is perfectly reliable (no bit errors, no packet loss).

Simple encapsulation. The sender sends data, and the receiver accepts it. No feedback is required.

Building Reliable Data Transfer Protocol

2. rdt 2.0: Channel with Bit Errors

Bits may be flipped (corrupted) during transmission.

Introduces Checksums for error detection and Feedback via ACKs (Positive Acknowledgement) and NAKs (Negative Acknowledgement).

If the receiver detects an error via the checksum, it sends a NAK, prompting the sender to retransmit the packet.

3. rdt 2.1 & 2.2: Handling Corrupted Feedback

What if the ACK/NAK itself is corrupted?

Introduces Sequence Numbers. By tagging packets with 0 or 1, the receiver can identify if an incoming packet is a new transmission or a duplicate caused by a corrupted ACK.

Building Reliable Data Transfer Protocol

3. rdt 3.0: Channel with Loss and Bit Errors

Packets or ACKs can be completely lost in the "lossy" medium.

Introduces Countdown Timers. If the sender does not receive an ACK within a specified time (the timeout period), it assumes the packet was lost and retransmits.

Building Reliable Data Transfer Protocol

1. **Error Detection (Checksums):** The sender computes a checksum over the payload. The receiver validates this to detect bit alterations caused by channel noise.
2. **Feedback (ACKs/NAKs):** Positive Acknowledgments (ACK) and Negative Acknowledgments (NAK) allow the receiver to signal the state of packet arrival.
3. **Sequence Numbers:** To resolve the "Ambiguous ACK" problem (where a sender cannot distinguish between a new packet and a retransmission), each packet is assigned a unique integer ID.
4. **Countdown Timers:** Necessary to recover from total packet loss. If a sender does not receive an ACK within a predefined Round Trip Time (RTT), it triggers an automatic retransmission.

Pipelining Protocol

A Pipelining Protocol is a reliable data transfer technique that allows a sender to transmit multiple packets without waiting for an acknowledgment (ACK) for each one.

This is a departure from the "Stop-and-Wait" model, where the sender remains idle until the receiver confirms the successful arrival of a single packet.

Two Major Approaches:

- Go-Back-N (GBN)
- Selective Repeat (SR).

Pipelining Protocol: Go Back-N (GBN)

GBN allows the sender to have multiple packets in the pipeline but maintains a simple, stateless receiver.

Sender Window: The sender can have up to N unacknowledged packets in the pipeline.

Cumulative ACK: An ACK with sequence number n indicates that all packets up to and including n have been received correctly.

Receiver Logic: The receiver only accepts packets in the correct order. If packet k is missing, all subsequent packets ($k+1$, $k+2$, etc.) are discarded, and the receiver re-sends an ACK for the last in-order packet.

Retransmission: If a timeout occurs for packet n , the sender retransmits packet n and all subsequent packets in the window.

Pipelining Protocol: Go Back-N (GBN)

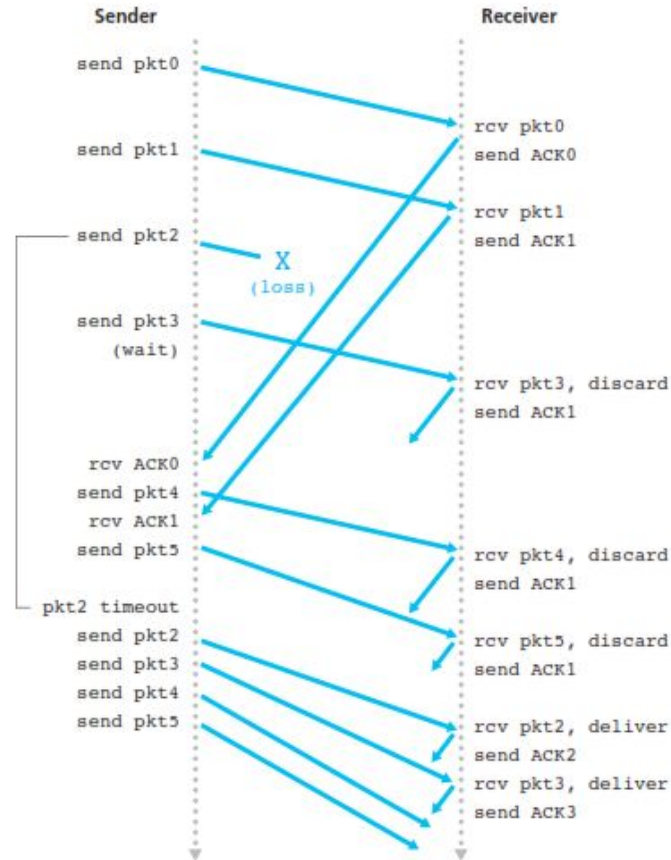


Figure 3.22 ♦ Go-Back-N in operation

Pipelining Protocol: Selective Repeat (SR)

SR improves efficiency over lossy links by only retransmitting the specific packets that were lost or corrupted.

Individual ACK: The receiver acknowledges each correctly received packet individually, regardless of whether it is in order.

Receiver Buffering: Out-of-order packets are stored in a buffer. Once the "gap" (the missing packet) is filled, the batch of packets is delivered to the upper layer.

Logical Timers: The sender maintains a separate timer for each unacknowledged packet.

Surgical Retransmission: When a timer for a specific packet expires, only that packet is retransmitted.

Constraint: To avoid overlap between new and old packets, the window size must be less than or equal to half the sequence number space.

Pipelining Protocol: Selective Repeat (SR)

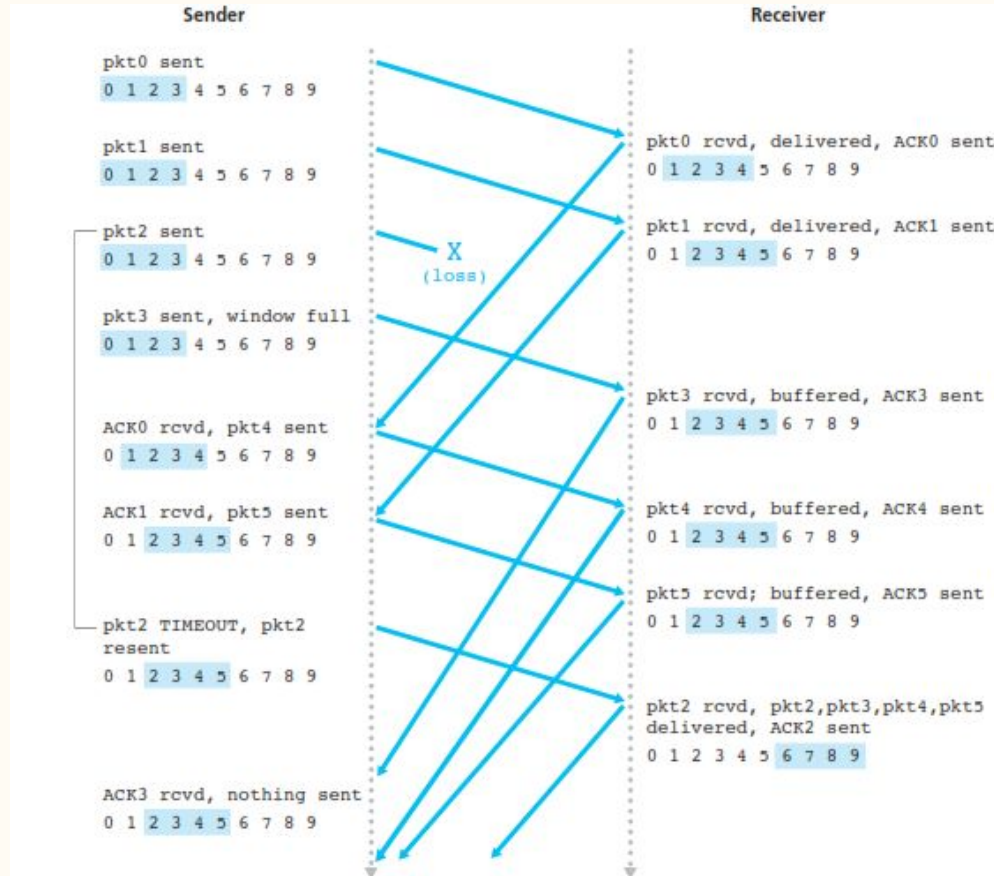


Figure 3.26 ♦ SR operation

The Transmission Control Protocol (TCP) Services

The Transmission Control Protocol (TCP) is a sophisticated, connection-oriented transport layer protocol designed to provide reliable, end-to-end delivery of data over an inherently unreliable network.

TCP acts as a bridge between the application layer and the network, offering a suite of services that simplify how software communicates across a distance.

Process-to-Process Delivery: TCP utilizes a set of 16-bit identifiers known as Port Numbers to ensure that data is delivered to the correct application on a host. This allows a single physical machine to run multiple services (like a web server and a mail server) simultaneously without cross-talk.

Stream-Oriented Communication: From the perspective of the application, TCP treats data as a continuous, unstructured stream of bytes. It is the responsibility of TCP to slice this stream into manageable chunks, called segments, for transmission across the network.

The Transmission Control Protocol (TCP) Services

Full-Duplex Data Flow: TCP supports a bidirectional flow of information, meaning data can be sent and received at the exact same time over a single connection. This is achieved by maintaining independent sequence numbers and buffers for both directions of the conversation.

Connection-Oriented Setup: No data is exchanged until a logical handshake is completed. This process ensures that both the sender and receiver are synchronized and have allocated the necessary memory resources to handle the upcoming traffic.

Transmission Control Protocol (TCP) Features

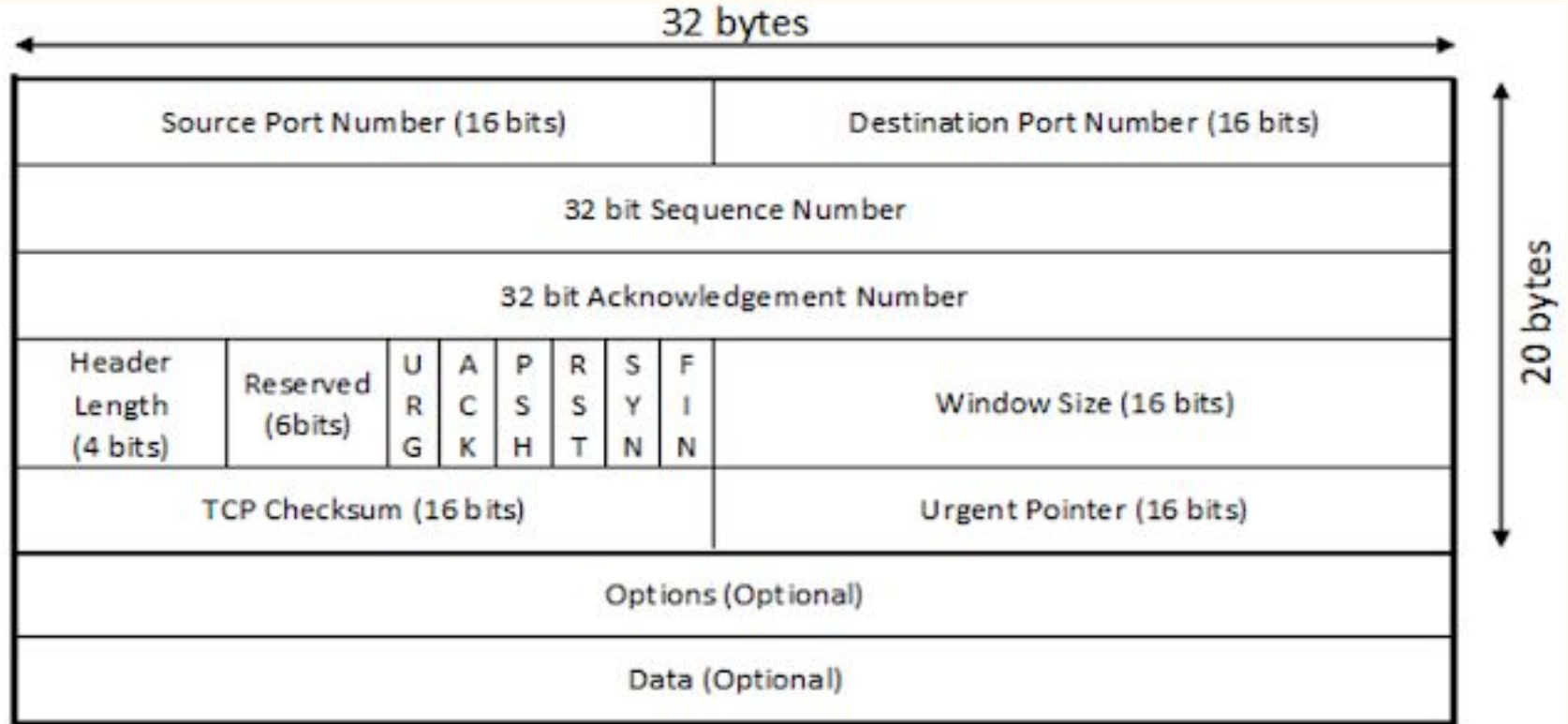
Reliability through Positive Acknowledgment: TCP ensures every byte is accounted for by requiring the receiver to send back an acknowledgment (ACK). If a segment is lost or corrupted in transit, the sender identifies the silence as a failure and retransmits the missing data.

Flow Control via the Sliding Window: To prevent a fast sender from overwhelming a slow receiver, TCP implements a Sliding Window mechanism. The receiver explicitly communicates how much buffer space it has left, and the sender is strictly forbidden from exceeding that limit.

Congestion Control Algorithms: TCP is a "good network citizen." It constantly monitors the network for signs of bottlenecking. If it detects packet loss, it assumes the network is congested and voluntarily reduces its transmission rate to help clear the path.

Error Detection and Ordering: Every segment includes a checksum to verify that bits were not flipped during transmission. Furthermore, because packets can take different routes and arrive out of sequence, TCP uses sequence numbers to reassemble the data in its original, correct order before passing it to the application.

Transmission Control Protocol (TCP) Header



Transmission Control Protocol (TCP) Header

Source and Destination Ports (16 bits each): These fields define the specific "doorways" at the source and destination hosts that the data is passing through.

Sequence Number (32 bits): This field tracks the position of the first byte of data in the current segment within the larger data stream, ensuring the receiver can reconstruct the stream perfectly.

Acknowledgment Number (32 bits): This communicates the sequence number of the next byte the receiver is waiting for, acting as a confirmation of all bytes received prior to that number.

Header Length (4 bits): This specifies where the header ends and the actual data begins, which is necessary because the header size can vary depending on the "Options" field.

Transmission Control Protocol (TCP) Header

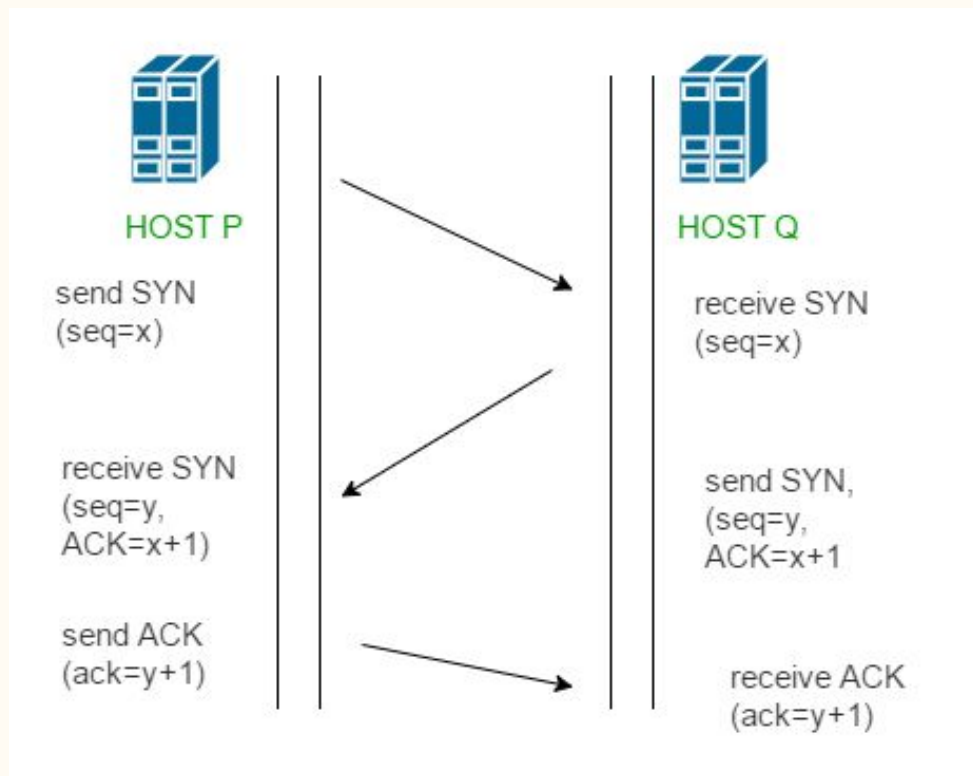
Control Flags (6 bits): These function as the "on/off" switches for the protocol's logic:

- SYN: Initiates the connection.
- ACK: Validates that the Acknowledgment field contains useful information.
- FIN: Gracefully closes the connection once data transfer is complete.
- RST: Immediately aborts a connection due to an unrecoverable error.

Window Size (16 bits): This is the "throttle" for flow control, indicating exactly how many bytes the receiver can currently accept in its buffer.

Checksum (16 bits): A mandatory field used to verify the integrity of both the header and the payload.

TCP 3-Way Handshake



TCP 3-Way Handshake

Step 1 (SYN): In the first step, the client wants to establish a connection with a server, so it sends a segment with SYN(Synchronize Sequence Number) which informs the server that the client is likely to start communication and with what sequence number it starts segments with

Step 2 (SYN + ACK): Server responds to the client request with SYN-ACK signal bits set. Acknowledgement(ACK) signifies the response of the segment it received and SYN signifies with what sequence number it is likely to start the segments with

Step 3 (ACK): In the final part client acknowledges the response of the server and they both establish a reliable connection with which they will start the actual data transfer.

TCP vs UDP

TCP	UDP
It is a connection-oriented protocol.	It is a connectionless protocol.
TCP reads data as streams of bytes, and the message is transmitted to segment boundaries.	UDP messages contain packets that were sent one by one. It also checks for integrity at the arrival time.
TCP messages make their way across the internet from one computer to another.	It is not connection-based, so one program can send lots of packets to another.
TCP rearranges data packets in the specific order.	UDP protocol has no fixed order because all packets are independent of each other.
The speed for TCP is slower.	UDP is faster as error recovery is not attempted.
Header size is 20 bytes	Header size is 8 bytes.
TCP is heavy-weight. TCP needs three packets to set up a socket connection before any user data can be sent.	UDP is lightweight. There are no tracking connections, ordering of messages, etc.

TCP vs UDP

TCP does error checking and also makes error recovery.	UDP performs error checking, but it discards erroneous packets.
Acknowledgment segments	No Acknowledgment segments
Using handshake protocol like SYN, SYN-ACK, ACK	No handshake (so connectionless protocol)
TCP is reliable as it guarantees delivery of data to the destination router.	The delivery of data to the destination can't be guaranteed in UDP.
TCP offers extensive error checking mechanisms because it provides flow control and acknowledgment of data.	UDP has just a single error checking mechanism which is used for checksums.

Network Congestion

Network congestion occurs when the total load placed on the network, the number of packets injected by hosts exceeds the maximum carrying capacity of the network resources, such as links and router buffers.

When too many packets are pumped into the system simultaneously, the infrastructure cannot process them fast enough, leading to a degradation of performance.

As long as the number of packets dumped into the subnet is within its capacity, they are delivered efficiently. However, once traffic increases beyond a critical threshold, routers are no longer able to cope and begin losing packets.

When packets are lost or delayed, senders often retransmit them, which adds even more traffic to the already overwhelmed network.

Congestion Control

Congestion control refers to the set of mechanisms and techniques used to regulate the traffic load injected into the network to ensure it stays below the network's capacity.

It is a global issue involving the cooperation of all hosts and routers to maintain the stability of the entire network fabric.

Congestion Control vs. Flow Control:

Flow Control: Manages the point-to-point traffic between a single sender and a single receiver to prevent the receiver from being overwhelmed.

Congestion Control: Manages the aggregate traffic entering the network to prevent the communication infrastructure itself from being overwhelmed.

Congestion Control

Open-Loop Congestion Control (Prevention): These mechanisms focus on prevention. They use policies applied at the source or destination to ensure that congestion does not happen in the first place. Once the system is running, these protocols do not typically change their behavior based on the current state of the network.

Closed-Loop Congestion Control (Removal): These are reactive mechanisms that treat congestion after it has already occurred. They rely on a feedback loop to detect congestion and send signals back to the source to reduce or remove the load.

Open-Loop Congestion Control

1. Retransmission Policy:

Retransmission is sometimes unavoidable when a packet is lost or corrupted. However, a "greedy" retransmission policy can actually trigger or worsen congestion. A good policy optimizes retransmission timers to ensure that a sender does not flood the network with duplicate packets unnecessarily.

2. Window Policy:

The type of sliding window used by the sender significantly impacts network load.

Selective Repeat (SR): This is the preferred window policy for congestion control. It only resends the specific packets that were lost.

Go-Back-N (GBN): This can be harmful during congestion because if one packet times out, the sender resends several packets (the entire window), even if most of them arrived safely. This creates redundant traffic that wastes bandwidth.

Open-Loop Congestion Control

3. Acknowledgment (ACK) Policy:

The receiver can directly influence the sender's speed. By implementing a policy where it does not acknowledge every single packet (e.g., acknowledging only every N th packet or using a timer), the receiver imposes less load on the network. Since ACKs are also packets that consume bandwidth, sending fewer of them helps keep the total load lower.

4. Discarding Policy:

Smart discarding by routers can prevent a complete system crash. In certain types of traffic, such as real-time audio, a router can be programmed to discard "less sensitive" packets if a buffer starts to fill up. This preserves the integrity of the overall transmission while preventing the queue from overflowing.

5. Admission Policy:

Primarily used in virtual-circuit networks, this functions as a Quality of Service (QoS) gatekeeper. Before a new connection is established, the switches check if they have enough resources to handle the new flow. If the network is already congested or if the new flow might cause congestion, the router can simply deny the connection request.

Closed-Loop Congestion Control

1. Back-pressure:

This is a hop-by-hop technique used primarily in Virtual-Circuit networks.

When a node (router) becomes congested, it stops receiving data from its immediate upstream neighbor. This pressure forces the upstream node to slow down, which may then cause its upstream neighbor to become congested, and so on.

The warning signal propagates backward, link-by-link, against the flow of data, eventually reaching the original source. This effectively throttles the source until the network can clear the backlog.

This method is impossible in datagram-based networks (like the modern Internet) because routers in those networks do not maintain state information about the specific upstream sender of a packet.

Closed-Loop Congestion Control

2. Choke Packets:

In this method, a congested node sends a specialized "choke packet" directly back to the source station.

The router that is overwhelmed by data generates a choke packet to inform the source that it needs to reduce its transmission rate.

Unlike back-pressure, the choke packet method does not necessarily affect intermediate nodes. The packet travels directly to the source, and intermediate nodes simply forward it without needing to throttle their own traffic.

Example: In the Internet, this is often implemented via the ICMP "Source Quench" message, which notifies a host that its datagrams are being discarded due to congestion at a router along the path.

Closed-Loop Congestion Control

3. Implicit Signaling:

In implicit signaling, there is no direct communication between the congested router and the source. The source must "guess" that the network is congested by interpreting indirect symptoms.

If a source sends several packets and does not receive acknowledgments (ACKs) within a reasonable time, it assumes a network bottleneck exists.

The observed delay in receiving ACKs or the occurrence of packet timeouts acts as a signal to the source that it must slow down its transmission rate to help alleviate the hidden congestion.

Closed-Loop Congestion Control

4. Explicit Signaling:

This method uses explicit bits within the data packets themselves to communicate congestion. This is faster and more direct than waiting for timeouts.

Backward Signaling: A bit is set in a packet moving in the direction opposite to the congestion (often in an ACK packet). This explicitly warns the source that it is injecting too much traffic and should throttle back.

Forward Signaling: A bit is set in a packet moving in the same direction as the congestion. This warns the destination that the network is struggling. The destination can then respond by intentionally delaying its own acknowledgments, which effectively forces the sender to slow down.

Traffic Shaping

Traffic shaping is a proactive congestion control technique used to regulate the rate and volume of data sent to the network.

Two techniques can shape traffic:

- 1. Leaky bucket**
- 2. Token bucket**

Traffic Shaping: Leaky Bucket

The Leaky Bucket algorithm is designed to convert a bursty, unpredictable stream of data into a strictly constant and smooth output flow.

The Analogy: Imagine a bucket with a small, fixed hole at the bottom. You can pour water into the bucket in large, sudden splashes (bursty data), but the water will only leak out of the hole at a steady, uniform rate.

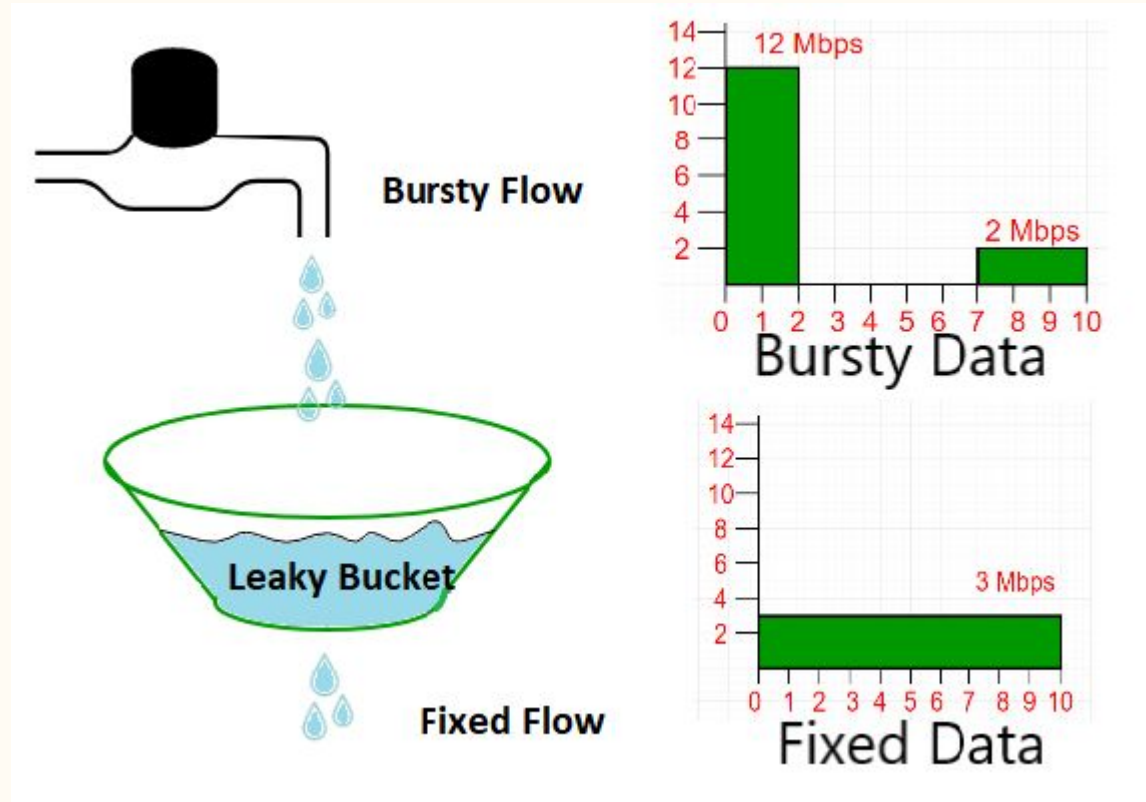
The host sends packets into a FIFO (First-In, First-Out) queue, which acts as the "bucket."

Regardless of how fast or inconsistently the packets arrive in the queue, they are pulled out and transmitted into the network at a rigid, average rate (represented as p)

If the host sends a burst that is larger than the capacity of the queue (the bucket is full), any additional incoming packets are immediately discarded.

Traffic Shaping: Leaky Bucket

It a rigid output rate and does not allow for any bursts to pass through to the network.



Traffic Shaping: Token Bucket

While the Leaky Bucket is effective, it is often seen as too restrictive for modern applications that naturally require occasional high-speed bursts.

The bucket holds "tokens" rather than actual data packets. These tokens are generated by a clock at a fixed rate of one token every Δt seconds.

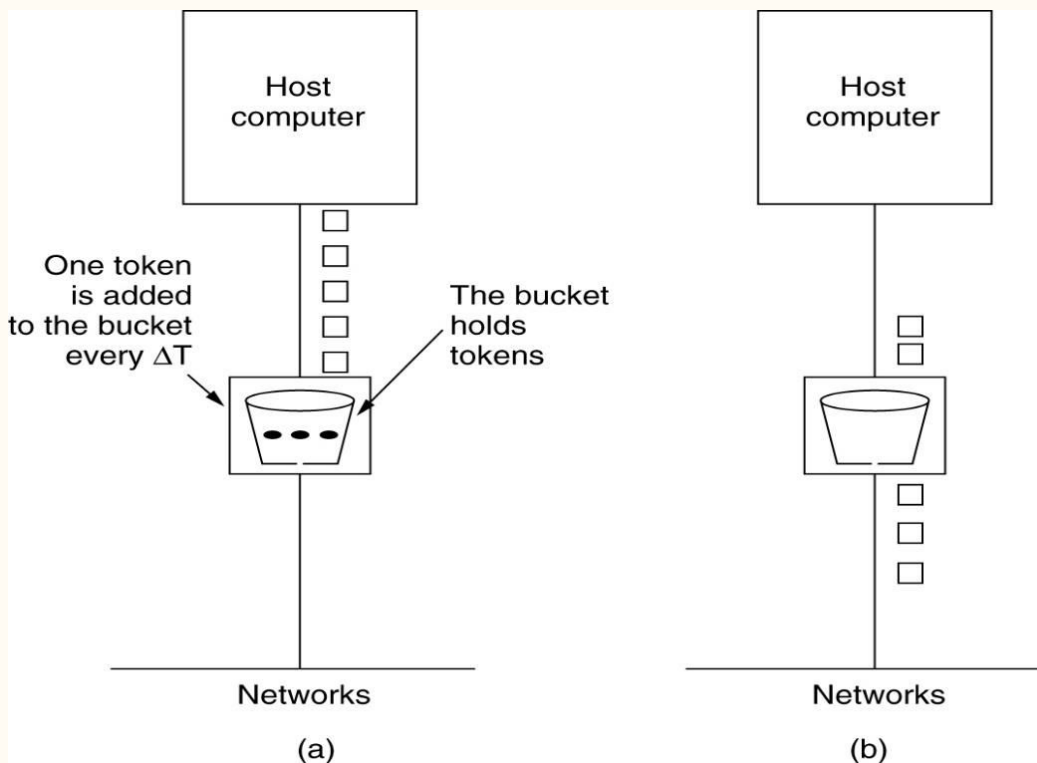
To transmit a packet, the host must first "capture and destroy" a token from the bucket.

If the bucket is empty, the host must wait for a new token to be generated before it can send the next packet.

If a host has been idle for a while, the bucket fills up with tokens (up to a predefined maximum capacity). This allows the host to send a large burst of data nearly instantaneously by consuming all the accumulated tokens at once.

Traffic Shaping: Token Bucket

Unlike the Leaky Bucket which drops packets, the Token Bucket drops tokens when the bucket is full. Packets are held in a separate data buffer until a token becomes available.



Token Bucket vs Leaky Bucket

Feature	Token Bucket Algorithm	Leaky Bucket Algorithm
Traffic Handling	Allows for burst traffic by saving tokens	Smooths traffic by enforcing a constant rate
Token/Bucket Concept	Uses tokens to control data transmission	Uses a leaky bucket to control data transmission
Data Dropping	Data waits until enough tokens are available	Excess data is discarded if the bucket overflows
Rate Flexibility	More flexible with bursts	More rigid, enforces a steady rate
Use Case	Suitable for bursty traffic and rate limiting	Suitable for steady rate enforcement and shaping
Implementation Complexity	Generally more complex	Generally simpler

The End